

UT-13: MANTENIMIENTO DE LA PERSISTENCIA DE LOS OBJETOS

¿Qué vamos a ver?

- Bases de datos orientadas a objetos. Características.
- Instalación del gestor de bases de datos.
- Creación de bases de datos.
- Mecanismos de consulta.
- El lenguaje de consultas: sintaxis, expresiones, operadores.
- Recuperación, modificación y borrado de información.

1. BASES DE DATOS ORIENTADAS A OBJETOS.

CARACTERÍSTICAS

Con el auge de la programación orientada a objetos aparece el modelo orientado a objetos, que permite establecer relaciones entre objetos y atributos, en lugar de hacerlo entre campos individuales.

El objetivo de este modelo es cubrir las limitaciones del modelo relacional. Con este modelo se incorporan mejoras como la herencia entre tablas, los tipos definidos por el usuario, etc.

Los objetos similares se agrupan en una clase y cada objeto de una clase particular se llama su instancia. Las clases permiten que un programador defina datos que no están incluidos en el programa. Dado que una clase solo define los datos que necesita, si se ejecuta un objeto de esa clase, no podrá acceder a otros datos, evitando así la corrupción de datos y garantizando la seguridad.

Las clases intercambian datos entre sí mediante el uso de mensajes llamados métodos. Tienen una propiedad llamada herencia, lo que significa que, si se define una clase, una subclase puede heredar sus propiedades sin definir sus propios métodos. Esto significa que una subclase puede implementar el mismo código. Esto acelera el desarrollo del programa.

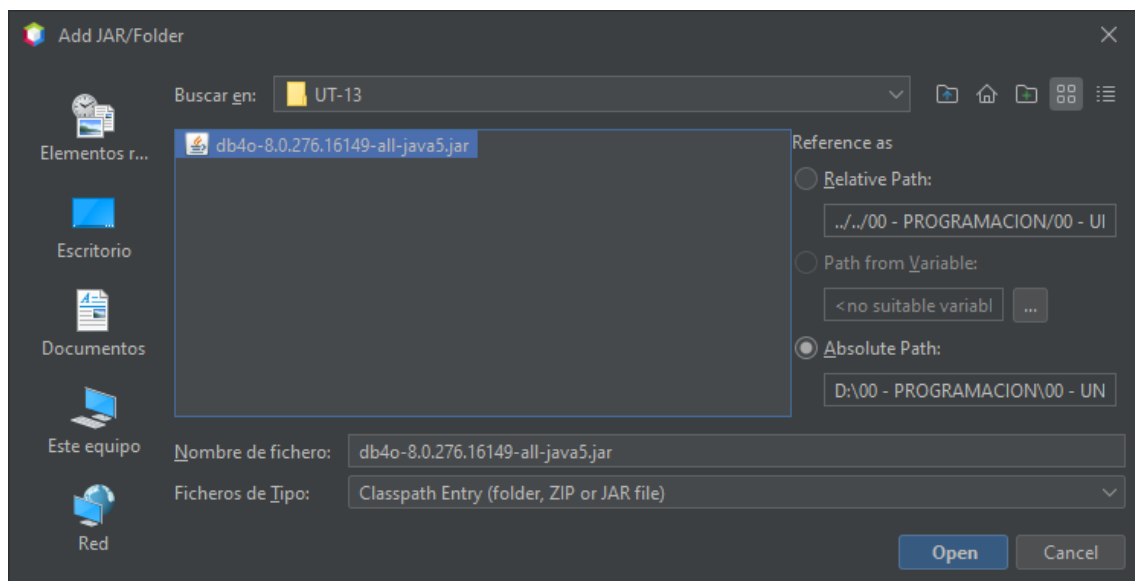
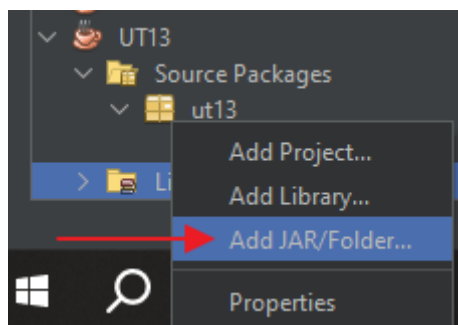
Las clases permiten agrupar objetos con características similares. Se puede crear una superclase combinando todas las clases. Esto conduce a una reducción de la redundancia de datos y la reutilización de clases, lo que permite un mantenimiento más fácil de los datos.

Puede que no sea necesario un lenguaje de consulta, ya que todas las transacciones tienen lugar al acceder a los objetos de manera transparente. Y, además, la base de datos es capaz de almacenar diferentes tipos de datos, como audio, video, imágenes, etc.

2. INSTALACIÓN DEL GESTOR DE BASES DE DATOS

En nuestro caso vamos a utilizar una base de datos orientada a objetos llamada **db4o**. Se trata de una base de datos muy ligera y que únicamente necesitaremos incluir una librería en nuestro proyecto para poder utilizarla y crear y manipular nuestras propias bases de datos orientadas a objetos.

Para añadir esta librería pulsaremos botón derecho sobre la carpeta Libraries de nuestro proyecto, seleccionaremos Add JAR/Folder, y con el navegador indicaremos el fichero que corresponde a la versión 8.0.276 de esta librería.



Una vez que tengamos añadida la librería de db4o, necesitaremos realizar 3 importaciones en nuestro proyecto, que serán las siguientes:

```
import com.db4o.Db4oEmbedded;  
import com.db4o.ObjectContainer;  
import com.db4o.ObjectSet;
```

Y con esto ya estaremos preparados para utilizar la base de datos orientada a objetos.

3. CREACIÓN DE BASE DE DATOS

Para crear una base de datos db4o solo necesitaremos ejecutar un método en nuestro código:

```
Db4oEmbedded.openFile(nombreFicheroBaseDatos);
```

Con `openFile` crearemos una base de datos nueva si el fichero no existe, o abriremos una existente si ya existe.

El resultado de este método lo almacenaremos en un objeto de tipo `ObjectContainer`, que será algo similar a la conexión de la base de datos.

Para cerrar esta conexión utilizaremos el método `close()`.

Veamos un pequeño ejemplo:

```
import com.db4o.Db4oEmbedded;
import com.db4o.ObjectContainer;
import com.db4o.ObjectSet;

public class UT13 {

    public static void main(String[] args) {

        ObjectContainer oc = null;

        try {
            oc = Db4oEmbedded.openFile(ruta);

        } catch (Exception e) {
            System.out.println(e.toString());
        }
        finally{
            try {
                if(oc != null)
                    oc.close();
            } catch (Exception e) {
                System.out.println(e.toString());
            }
        }
    }
}
```

```
}  
}  
}
```

En el caso de que queramos eliminar la base de datos, será tan sencillo como eliminarlo como si fuese un fichero más, es decir:

```
File f = new File("personas.db4o");  
f.delete();
```

A la hora de añadir objetos a la base de datos, primero tendremos que crear dicho objeto y, después, se lo pasaremos como parámetro al método store(T objeto) del objeto ObjectContainer.

```
import com.db4o.Db4oEmbedded;  
import com.db4o.ObjectContainer;  
import com.db4o.ObjectSet;  
  
public class UT13 {  
  
    public static void main(String[] args) {  
  
        ObjectContainer oc = null;  
  
        try {  
            //Creación o apertura de la base de datos  
            ObjectContainer oc =  
Db4oEmbedded.openFile("personas.db4o");  
  
            //Insertar objetos  
            Persona p1 = new Persona("Fernando", 30, 1.73, 73.0);  
            Persona p2 = new Persona("Pepe", 30, 1.75, 80.0);  
            Persona p3 = new Persona("Alfredo", 20, 1.9, 90.0);  
            Persona p4 = new Persona("Roberto", 35, 1.70, 70.0);  
            Persona p5 = new Persona("Domingo", 30, 1.73, 80.0);  
            oc.store(p1);  
            oc.store(p2);  
            oc.store(p3);  
            oc.store(p4);  
            oc.store(p5);  
        }  
    }  
}
```

```
    } catch (Exception e) {  
        System.out.println(e.toString());  
    }  
    finally{  
        try {  
            if(oc != null)  
                oc.close();  
        } catch (Exception e) {  
            System.out.println(e.toString());  
        }  
    }  
}  
}
```

4. MECANISMOS DE CONSULTA

Db4o cuenta con tres formas distintas de realizar consultas:

- S.O.D.A. (Simple Object Database Access) / Acceso Simple a Bases de Datos de Objetos.
- Q.B.E. (Query By Example) / Consulta por Ejemplo o Plantilla.
- N.Q. (Native Queries) / Consultas Nativas.

Nosotros únicamente vamos a aprender a realizar consultas con Q.B.E.

5. EL LENGUAJE DE CONSULTAS: SINTAXIS, EXPRESIONES, OPERADORES.

Para realizar consultas por medio de Q.B.E. primero vamos a necesitar crearnos un objeto del tipo que estamos buscando y que nos sirva de plantilla, o ejemplo, para localizar los objetos que deseamos.

Es muy conveniente que al menos tengamos 2 constructores de la clase con la que vamos a trabajar en la base de datos, uno vacío y otro con todos los

parámetros correspondientes a los atributos. También es aconsejable que no utilicemos tipos primitivos, sino que utilicemos sus correspondientes Wrapper.

```
public class Persona {  
    private String nombre;  
    private Integer edad;  
    private Double altura;  
    private Double peso;  
  
    public Persona(){}  
  
    public Persona(String nombre, Integer edad, Double altura, Double  
peso) {  
        this.nombre = nombre;  
        this.edad = edad;  
        this.altura = altura;  
        this.peso = peso;  
    }  
}
```

Cuando nosotros necesitemos recuperar todos los objetos persona de una base de datos orientada a objetos, nos crearemos un objeto con el constructor vacío, que nos servirá de plantilla.

```
Persona p = new Persona();
```

En el caso de que necesitemos que los objetos cumplan una condición relacionada con sus atributos, utilizaremos el constructor con todos los atributos, rellenando el que nos interesa y poniendo el resto a null. Esto lo podremos hacer con un solo atributo o con varios, pero en el caso de que utilicemos varios. Debemos tener en cuenta que en ese caso todos los objetos que obtendremos cumplirán todas las condiciones y no solo alguna de ellas.

```
Persona p = new Persona("Fernando", null, null, null);
```

```
Persona p = new Persona(null, null, 1.73, 80.0);
```

6. RECUPERACIÓN, MODIFICADO Y BORRADO DE INFORMACIÓN

6.1. Recuperación de la información

Cuando necesitamos recuperar uno o varios objetos de la base de datos orientada a objetos vamos a hacer uso del método `queryByExample(T objeto)` del objeto `ObjectContainer`.

```
ObjectSet<T> queryByExample(T objeto);
```

Como parámetro pasaremos el objeto que vamos a utilizar como ejemplo, o plantilla.

En el caso de necesitar recuperar todos los objetos de un tipo determinado, crearemos un objeto con el constructor vacío y lo pasaremos por parámetro.

Si lo que necesitamos es que cumplan una serie de condiciones, utilizaremos el constructor con los datos referentes a las condiciones y el resto a null.

Como resultado `queryByExample` devolverá un `ObjectSet` del mismo tipo que el objeto utilizado como plantilla. El `ObjectSet` agrupará ninguno, uno o varios objetos.

Haremos uso de un bucle para recorrer todos los resultados obtenidos. La condición de ruptura del bucle nos la dará el método `hasNext()` del `ObjectSet`, ya que en el momento en el que no queden objetos nos devolverá un "false".

Para obtener siguiente objeto de un `ObjectSet` utilizaremos el método `next()`, que nos devolverá un objeto del tipo que esperamos.

Veamos un ejemplo simple:

```
public class UT13 {  
  
    public static void main(String[] args) {  
  
        ObjectContainer oc = null;  
  
        try {  
            //Creación o apertura de la base de datos
```



```

        ObjectContainer oc =
Db4oEmbedded.openFile("personas.db4o");

        Persona p = new Persona();

        ObjectSet<Persona> result = oc.queryByExample(p);

        p = new Persona("Domingo", null, null, null);
        result = oc.queryByExample(p);
        while(result.hasNext()){
            Persona per = result.next();
            System.out.println(per);
        }

        System.out.println("");
        p = new Persona();
        result = oc.queryByExample(p);
        while(result.hasNext()){
            Persona per = result.next();
            System.out.println(per);
        }

    } catch (Exception e) {
        System.out.println(e.toString());
    }
    finally{
        try {
            if(oc != null)
                oc.close();
        } catch (Exception e) {
            System.out.println(e.toString());
        }
    }
}
}

```

```
Persona{nombre=Domingo, edad=30, altura=1.73, peso=80.0}
```

```
Persona{nombre=Fernando, edad=30, altura=1.73, peso=73.0}
```

```
Persona{nombre=Pepe, edad=30, altura=1.75, peso=80.0}
```

```
Persona{nombre=Alfredo, edad=20, altura=1.9, peso=90.0}
```

```
Persona{nombre=Roberto, edad=35, altura=1.7, peso=70.0}
```

```
Persona{nombre=Domingo, edad=30, altura=1.73, peso=80.0}
```

6.2. Modificación de información

Para modificar un objeto que tenemos almacenado en la base de datos, primero debemos recuperarlo de la misma, le realizaremos las modificaciones que

necesitemos y lo volvemos a almacenar con el método `store(T Objecto)`, que ya conocemos.

La misma base de datos se encargará de reconocer que ese objeto ya existe y lo sobrescribirá.

Veamos un ejemplo:

```
public class UT13 {

    public static void main(String[] args) {

        ObjectContainer oc = null;

        try {

            //Creación o apertura de la base de datos
            ObjectContainer oc =
Db4oEmbedded.openFile("personas.db4o");

            Persona p = new Persona();

            ObjectSet<Persona> result = oc.queryByExample(p);

            p = new Persona("Domingo", null, null, null);
            result = oc.queryByExample(p);
            while(result.hasNext()){
                Persona per = result.next();
                System.out.println(per);
                per.setAltura(1.75);
                oc.store(per);
            }

            System.out.println("");
            p = new Persona("Domingo", null, null, null);
            result = oc.queryByExample(p);
            while(result.hasNext()){
                Persona per = result.next();
                System.out.println(per);
            }

        } catch (Exception e) {
            System.out.println(e.toString());
        }
        finally{
            try {
                if(oc != null)
                    oc.close();
            }
        }
    }
}
```

```
        } catch (Exception e) {  
            System.out.println(e.toString());  
        }  
    }  
}
```

```
Persona{nombre=Domingo, edad=30, altura=1.73, peso=80.0}
```

```
Persona{nombre=Domingo, edad=30, altura=1.75, peso=80.0}
```

6.3. Eliminación de la información

En el caso de la eliminación operaremos de una forma similar a la modificación, en primer lugar extraeremos el objeto u objetos que queremos eliminar y luego se lo pasaremos como parámetro al método delete(T objeto) del objeto ObjectContainer.

Veamos un ejemplo:

```
public class UT13 {  
  
    public static void main(String[] args) {  
  
        ObjectContainer oc = null;  
  
        try {  
  
            ObjectContainer oc = Db4oEmbedded.openFile("personas.db4o");  
  
            Persona p = new Persona();  
  
            ObjectSet<Persona> result = oc.queryByExample(p);  
  
            p = new Persona();  
            result = oc.queryByExample(p);  
            while(result.hasNext()){  
                Persona per = result.next();  
                System.out.println(per);  
            }  
  
            p = new Persona(null, 30, null, 80.0);  
            result = oc.queryByExample(p);  
            while(result.hasNext()){  
                Persona per = result.next();  
                oc.delete(per);  
            }  
        }  
    }  
}
```

```
    }

    System.out.println("");
    p = new Persona();
    result = oc.queryByExample(p);
    while(result.hasNext()){
        Persona per = result.next();
        System.out.println(per);
    }

} catch (Exception e) {
    System.out.println(e.toString());
}
finally{
    try {
        if(oc != null)
            oc.close();
    } catch (Exception e) {
        System.out.println(e.toString());
    }
}
}
```

```
Persona{nombre=Fernando, edad=30, altura=1.73, peso=73.0}
Persona{nombre=Pepe, edad=30, altura=1.75, peso=80.0}
Persona{nombre=Alfredo, edad=20, altura=1.9, peso=90.0}
Persona{nombre=Roberto, edad=35, altura=1.7, peso=70.0}
Persona{nombre=Domingo, edad=30, altura=1.75, peso=80.0}
```

```
Persona{nombre=Fernando, edad=30, altura=1.73, peso=73.0}
Persona{nombre=Alfredo, edad=20, altura=1.9, peso=90.0}
Persona{nombre=Roberto, edad=35, altura=1.7, peso=70.0}
```